

ECE/CS 552: INTRODUCTION TO COMPUTER ARCHITECTURE

Project Description – Phase 2

Due on Monday, November 18, 2024, 11:59pm

In Phase 2 of this project, the task is to design a 5-stage pipelined processor to implement the WISC-F24 ISA.

Functional blocks of the single-cycle realization in Phase 1 of this project should be reused when possible.

The main task in Phase 2 is the implementation of pipeline control. You will implement (a) control blocks for hazard (both control and data) detection and mitigation (stalls, flushes and forwarding), and (b) control and data path modifications for data forwarding, including register file bypassing. More details are described below.

Either Modelsim or Icarus should be used as the simulator to verify the design. **You are required to follow the Verilog rules as specified by the rules document uploaded on canvas. NOTE: the only exception to this rule is the required use of *inout* (tri-state logic) in the register file. Do not use *inout* anywhere else in your design.**

1. WISC-F24 ISA Summary [unchanged from Phase 1]

This is a summary of the ISA from section 1 of Phase 1 Project Description. WISC-F24 contains a set of 16 instructions specified for a 16-bit data-path with load/store architecture.

The WISC-F24 memory is byte addressable, even though all accesses (instruction fetches, loads, stores) are restricted to half-word (2-byte), naturally-aligned accesses.

WISC-F24 has a register file, and a 3-bit FLAG register. The register file comprises sixteen 16-bit registers and has 2 read ports and 1 write port. Register \$0 is hardwired to 0x0000. The FLAG register contains three bits: Zero (Z), Overflow (V), and Sign (N).

The list of instructions and their opcodes are summarized in Table 1 below. Please refer to the Phase 1 handout for more details.

Table 1: Table of opcodes

Instruction	Opcode
ADD	0000
SUB	0001
XOR	0010
RED	0011
SLL	0100
SRA	0101
ROR	0110
PADDSB	0111
LW	1000
SW	1001
LLB	1010
LHB	1011
B	1100
BR	1101
PCS	1110
HLT	1111

2. Memory System [unchanged from Phase 1]

For this stage of the project, the memory modules are the same as in Phase 1. The processor will have separate single-cycle instruction and data memory, which are both byte-addressable. The instruction memory has a 16-bit address input and a 16-bit data output. The data memory has a 16-bit address input, a 16-bit data input, a 16-bit data output, and a write enable signal. If the write signal is asserted, the memory will write the data input bits to the location specified by the input address.

Verilog modules are provided for both memories.

The instruction memory contains the binary machine code instructions to be executed on your processor.

3. Implementation [new information]

3.1 Pipelined Design [new information]

Your design must use a **five-stage pipeline** (IF, ID, EX, MEM, WB) similar to that in the class material. The design will make use of Verilog modules that you developed as part of the homework assignments along with the modules developed for Phase 1 of the project. The pipeline register modules can use multiple instances of register.v from the Register File for register storage (ignore the 2nd read port), or you can choose to make one directly out of several dff.v instances. Don't worry about precise packing – if you waste some bits, that's fine.

You must implement hazard detection so that your pipeline correctly handles all dependences. You are required to implement register bypassing, EX-to-EX forwarding, MEM-to-EX forwarding, **and MEM-to-MEM forwarding as described in HW4**. You need to handle both data hazards and control hazards. You must implement predict-not-taken for branch instructions, which flushes instructions when branches are taken. You are not required to implement other optimizations to reduce branch delays such as dynamic branch prediction. Branches should be resolved in the ID stage. You do not need to implement data forwarding to branches in ID, but your hazard detection unit should insert (minimum) stall cycles to avoid these data hazards.

It is highly recommended to make a table that lists (for each instruction) the control signals needed at each pipeline stage. Also, make a table listing the input signals and output signals for the data and control hazard detection units (data hazard stalls and control hazard flushes). The data hazard detection unit should be aware of available data forwarding and register bypassing.

3.2 Stall/Flush [new information]

A stall is performed by using a global stall signal, which disables the write-enable signals to the D-FFs of the upstream pipeline registers (i.e., upstream from the stalling stage).

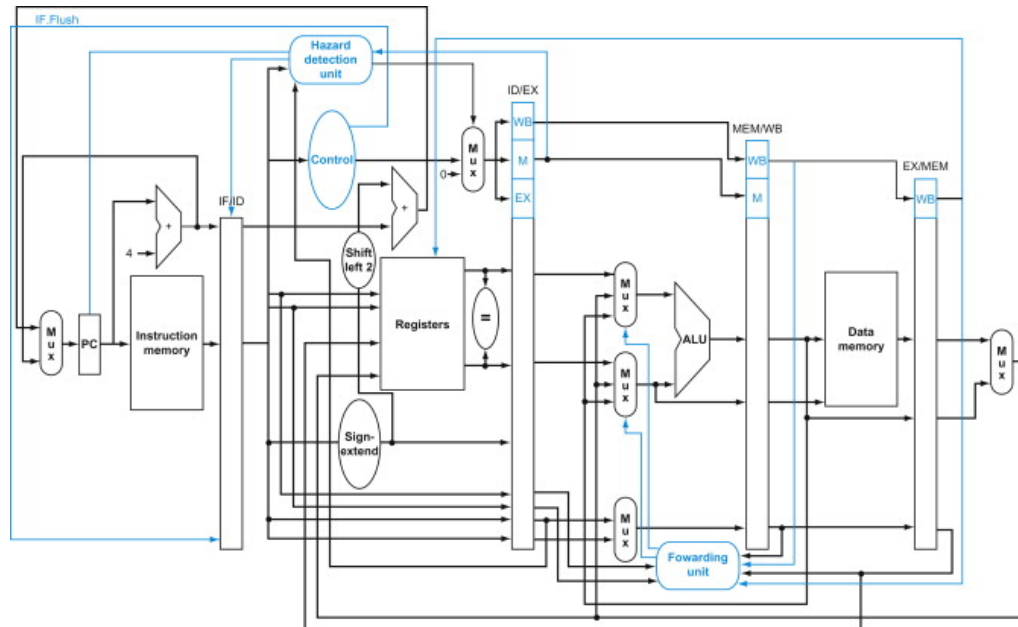
Flushes can be performed by converting the flushed operations into NOPs (i.e., preventing them from performing register/memory writes and from making changes to any other processor state, such as flags).

3.3 HLT Instruction [new information]

The HLT instruction should raise the 'hlt' signal only when it reaches the writeback stage. In the IF stage, you need to check for HLT instructions. If you fetch a HLT instruction, you must prevent the PC from being updated, thus stopping the program from fetching beyond the HLT instruction. The only exception is when the HLT instruction is fetched immediately after a taken branch: in this case, the HLT instruction is part of the wrong predict-not-taken branch path and will be flushed, thus the PC should be updated to the branch target address as usual.

3.4 Schematic [new information]

A summary diagram of the 5-stage pipeline is shown below (COD Figure 4.65):



Note: Not all hardware components and signals are shown here. There are more detailed diagrams of each stage of the pipeline in your textbook (along with details of how hazards are detected and resolved). You can refer to them for building the pipeline stages of the project.

3.5 Reset Sequence [unchanged from phase 1]

WISC-F24 has an active high reset input (*rst*). Instructions are executed when *rst* is low. If *rst* goes high for one clock cycle, the contents of the state of the machine is reset and starts execution at address 0x0000.

4. Interface [unchanged from phase 1]

Your top level Verilog code should be in file named *cpu.v*. It should have a simple 4-signal interface: *clk*, *rst*, *hlt* and *pc[15:0]*.

Signal Interface of <i>cpu.v</i>		
Signal:	Direction:	Description:
<i>clk</i>	in	System clock
<i>rst</i>	in	Active high reset. A 1'b1 on this signal resets the processor and causes execution to start at address 0x0000
<i>hlt</i>	out	When your processor encounters the HLT instruction, it will assert this signal once it has finished processing the last instruction before the HLT
<i>pc[15:0]</i>	out	PC value over the course of program execution

5. Submission Requirements [read carefully]

1. You were provided with an assembler in Phase 1 to convert your text-level test cases into machine level instructions. You will also be provided with a global testbench and test case. The test case should be run with the

testbench and the output (as a .txt file) should be submitted for Phase 2 evaluation. In other words, test1.list should produce test1.txt.

2. You are also required to submit a zipped file containing: all the Verilog files of your design, all testbenches used and any other support files.